

Functions and Pointers, Dynamic Memory Allocation

Parameter Passing

- Also called as Argument Passing
- Passing arguments from calling function/caller to called function/callee.
- Arguments in calling function known as actual or original arguments.
- Arguments in called function known as formal or dummy arguments.
- Types
 - Call by value or Pass by Value
 - Call by Reference or Pass by Reference

Call by Value or Pass by Value

- Its default mechanism of parameter passing
- Only copy of actual arguments are passed to formal arguments.
- The formal parameter is stored in the local data area of the callee.
- Any changes in formal arguments will not be reflected in actual arguments.

Calling Function

Assume

int a=10, b=20

Scale(a,b)

Print a and b values

a=10, b=20

Copy values
10, 20

Called Function

Scale(a,b) // a=10, b=20

a=a*5;

b=b+50;

Print a and b values

a=50, b=70

Swapping of two values

```
#include<stdio.h>
void swap(int x, int y)
{
    int t;
    printf("value of x and y in before swap %d and %d \n",x,y);
    t=x;    // t=10
    x=y;    // x=20
    y=t;    // y = 10
    printf("value of x and y in after swap %d and %d \n",x,y);
}
void main()
{
    int a,b;
    printf("Enter two integers");
    scanf("%d%d",&a,&b);
    swap(a,b);
    printf("value of a and b in main function %d and %d \n",a,b);
}
```

Run

Enter two integers

10

20

value of x and y in before swap 10 and 20

value of x and y in after swap 20 and 10

value of a and b in main function 10 and 20

Call by Reference or Pass by Reference

- While calling a function, instead of passing the values of variables, we pass address of variables (location of variables) to the function known as Call By Reference.
- Any changes in formal arguments will reflected in actual arguments.
- Pointer variables are necessary to define to store the address values of variables.

Calling Function

Assume

int a=10, b=20

Scale(&a,&b)

Print a and b values

a=50, b=70

Address
&a, &b



Called Function

Scale(*a,*b)

*a=(*a)*5;

*b=(*b)+50;

Print *a and *b values

a=50, b=70

Swapping of two values

```
#include<stdio.h>
void swap(int *x, int *y)
{
    int t;
    printf("value of x and y in before swap %d and %d \n",*x,*y);
    t=*x;    // t=10
    *x=*y;    // *x=20
    *y=t;    // *y = 10
    printf("value of x and y in after swap %d and %d \n",*x,*y);
}
void main()
{
    int a,b;
    printf("Enter two integers");
    scanf("%d%d",&a,&b);
    swap(&a,&b);
    printf("value of a and b in main function %d and %d \n",a,b);
}
```

Run

Enter two integers

10

20

value of x and y in before swap 10 and 20

value of x and y in after swap 20 and 10

value of a and b in main function 20 and 10

Sorting of Names using Pass by Reference

```
#include<stdio.h>
#include<string.h>
void ascend(int n, char *x[]);
void main()
{
    char *name[10];
    int i,n=0;
    printf("Enter number of Strings : ");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        name[i]=(char *)malloc(20*sizeof(char));
        scanf("%s",name[i]);
    }
}
```

Run:

Enter number of Strings : 3
sankar
ganesh
balaji


```

ascend(n,&name);
void ascend(int n, char *x[])
{
    int i,j;  char s[20];
    for(i=0;i<n-1;i++)
    {
        for(j=i+1;j<n;j++)
        {
            if(strcmp(x[i],x[j])>0)
            {
                strcpy(s,x[j]);
                strcpy(x[j],x[i]);
                strcpy(x[i],s);
            }
        }
    }
}

```

n=3, x=name // x pointing &name

x	[0]	[1]	[2]
	balaji	sankar	ganesh
	balaji	ganesh	sankar

i	i<2	j=i+1	j<3	strcmp(x[i],x[j])>0	Ascii Compare
1	1<2 ->T	2	2<3->T	(sankar, ganesh)>0	115-104=11>0 swap
		3	3<3->F		
2	2<2 ->F				


```

void ascend(int n, char *x[])
{
    int i,j;
    char s[20];
    for(i=0;i<n-1;i++)
    {
        for(j=i+1;j<n;j++)
        {
            if(strcmp(x[i],x[j])>0)
            {
                strcpy(s,x[j]);
                strcpy(x[j],x[i]);
                strcpy(x[i],s);
            }
        }
    }
}

```

The Ascending Order

balaji
ganesh
sankar

```

ascend(n,name); //ascend(n,&name)
printf("\nThe Ascending Order \n");
for(i=0;i<n;i++)
{
    printf("%s\n",name[i]);
}
}

```

Memory Allocation

- Allocate memory in RAM at compile time or run time.
- **Static and Dynamic memory allocation**

Static memory allocation	Dynamic memory allocation
Memory is allocated at compile time.	Memory is allocated at run time.
Memory can't be increased while executing program.	Memory can be increased while executing program.
Used in array	used in linked list

Function malloc()

- malloc means memory allocation
- Allocates single block of requested memory.
- It doesn't initialize memory at execution time, so it has garbage value initially.
- It doesn't clear the memory.
- It returns NULL if memory is not sufficient.
- It allocates memory of a specific size.
- This size is passed as parameter to it.
- This size is allocated from a heap.
- It performs its job quickly.

Function malloc()

- C library function `void *malloc(size_t size)` allocates the requested memory and returns a pointer to it.
`size` – This is the size of the memory block, in bytes
- `malloc()` returns a pointer to the allocated memory, or `NULL` if the request fails.
- The syntax of `malloc()` function is given below:
`ptr=(cast-type*)malloc(byte-size)`

Ex.

```
cpv=(char *) malloc(15); // 15 Bytes allocated  
ipv=(int *) malloc(10);  // 20 Bytes allocated  
fpv=(float *) malloc(10); // 40 Bytes allocated
```


Program for strings

```
#include<stdio.h>
#include<stdlib.h>
#include<alloc.h>
void main()
{
    char *name;
    name=(char *)malloc(6);
    printf("Enter your Name\n");
    scanf("%s",name);
```

Run:

Enter your Name
balaji

```
printf("Name: %s Address =%u\n",name,name);
printf("Address+5 = %u\n",name+5);
printf("Char at 5th Position is %c\n",*(name+5));
printf("Char at 8th Position is %c\n",*(name+8));
free(name);
printf("Name: %s Address =%u\n",name,name);
}
```

Name: balaji Address =1940
Address+5 = 1945
Char at 5th Position is i
Char at 8th Position is
Name: Address =1940

```
printf("Name: %s Address=%u\n",name,name);  
printf("Address+5 = %u\n",name+5);  
printf("Char at 5th Position is %c\n",*(name+5));  
printf("Char at 8th Position is %c\n",*(name+8));  
free(name);  
printf("Name: %s Address=%u\n",name,name);  
}
```

- free() function deallocates memory allocated by malloc, calloc or realloc.
- Syntax: free(pv);
Ex. free(name)

Name: balaji Address =1940
Address+5 = 1945
Char at 5th Position is i
Char at 8th Position is
Name: Address =1940

Program for integers

```
#include<stdio.h>
#include<stdlib.h>
#include<alloc.h>
void main()
{
    int *a,n,i;
    printf("Enter the size of the array\n");
    scanf("%d",&n);
    a=(int *)malloc(n);
```

Run:

Enter the size of the array
5

```
printf("Value of a is %u\n",a);
printf("Enter %d integer inputs\n",n);
for(i=0;i<n;i++)
    scanf("%d",a+i);
for(i=0;i<n;i++)
    printf("a[%d] value =%d Address=%u\n",i,*(a+i),(a+i));
printf("a[%d] value = %d\n",n+3,*(a+n+3));
free(a);
}
```

a[0] value =111 Address=1946
a[1] value =555 Address=1948
a[2] value =888 Address=1950
a[3] value =666 Address=1952
a[4] value =222 Address=1954
a[8] value = 0

Value of a is 1946
Enter 5 integer inputs
111
555
888
666
222

Function calloc()

- calloc means contiguous allocation
- It allocates multiple blocks of requested memory.
- It is a dynamic memory allocation function which is used to allocate the memory to complex data structures such as arrays and structures.
- It initially initialize all bytes to zero.
- It returns NULL if memory is not sufficient.
- It is slow in comparison to 'malloc' method.

- C library function **void *calloc(size_t nitems, size_t size)** allocates the requested memory and returns a pointer to it.

nitems – Number of elements to be allocated

size – Size of elements

- calloc() returns a pointer to the allocated memory, or NULL if the request fails.
- The syntax of calloc() function is given below:
ptr=(cast-type*)calloc(number, byte-size)

Ex.

cpv=(char *) calloc(15,sizeof(char)); // 15 Bytes allocated

ipv=(int *) calloc(10,sizeof(int)); // 20 Bytes allocated

fpv=(float *) calloc(15,sizeof(float)); // 60 Bytes allocated

Function realloc()

- realloc means re-allocation
- If memory is not sufficient for malloc() or calloc(), you can reallocate the memory by realloc() function.
- It changes the memory size.
- The syntax of realloc() function.

`ptr=realloc(ptr, new-size)`

`ptr` – The pointer which is pointing the previously allocated memory block by malloc or calloc

`new-size` – The new size of memory block

Program for strings

```
#include<stdio.h>
#include<stdlib.h>
#include<alloc.h>
void main()
{
    char *name;
    name=(char *)calloc(6,sizeof(char));
    printf("Enter your Name\n");
    scanf("%s",name);
```

Run:

Enter your Name
sankar


```
printf("Name: %s Address =%u\n",name,name);
printf("Char at 5th Position is %c\n",*(name+5));
printf("Char at 8th Position is %c\n",*(name+8));
name=(char *)realloc(name,12);
printf("Enter your Name\n");
scanf("%s",name);
printf("Name: %s Address =%u\n",name,name);
printf("Char at 8th Position is %c\n",*(name+8));
free(name);
}
```

Name: sankar Address =2018
Char at 5th Position is r
Char at 8th Position is
Enter your Name
sankarganesh
Name: sankarganesh Address =2028
Char at 8th Position is n

Dynamic memory allocation in C

- The concept of dynamic memory allocation in c language enables the C programmer to **allocate memory at runtime**. Dynamic memory allocation in c language is possible by 4 functions of stdlib.h header file.

1.malloc()

2.calloc()

3.realloc()

4.free()